



Innleveringsfrist: 2026-01-18

Mål for denne øvingen:

Overføre programmeringskonsepter vi kjenner til fra Python til C++.

Generelle krav:

- Bruk de eksakte navn og spesifikasjoner gitt i oppgaven.
- 70% av øvingen må godkjennes for at den skal vurderes som bestått.
- Øvingen skal leveres på **INGInious**.
- Benytt VS Code til å skrive, compilere og kjøre kode. Oppsett og installasjon av VS Code, og verktøy nødvendige for å følge dette øvingsopplegget er beskrevet i øving 0.

Innledning

I denne øvingen skal vi demonstrere likheter og ulikheter mellom C++ og Python. Oppgavene vil i stor grad gå ut på å oversette Python-kode til C++, og vil også i noen grad gi en slags oppfriskning av ITGK-kunnskapene. Hovedhensikten er å overføre kunnskaper om Python over til C++. Du vil også lære om forskjeller mellom de to språkene, og om noen fallgruver. Merk at Python-versjonen som brukes i eksemplene er Python 3.x, som benyttes i ITGK. Dersom du er vant med Python 2.7 vil det være visse **forskjeller**, blant annet fungerer divisjon ulikt.

Lynintroduksjon til forskjellene mellom Python og C++

main-funksjonen

Når du kjører et C++ program er det koden i `main`-funksjonen som blir kjørt. Du kan bare ha *én* `main`-funksjon i prosjektet ditt. Hvis du har flere vil ikke programmet kompilere. `main`-funksjonen kalles ofte programmets hovedfunksjon, og ligger som regel i filen `main.cpp`.

Utskrift til skjerm

I Python brukes funksjonen `print(a,b,c)` for å skrive ut tekst til skjerm. I C++ sender vi i stedet tekststrenger og andre objekter vi ønsker å skrive til skjerm til objektet `cout` ("character output", også kalt "standard output stream") ved å bruke `<<`-operatoren. Det er tillatt å nøste den og dermed skrive ut flere objekter om gangen. Siste linje i figuren under gir et eksempel på det.

Python sin `print`-funksjon legger til linjeskift på slutten som standard. I C++ må dette gjøres eksplisitt. Både i Python og i C++ kan `'\n'` brukes til å eksplisitt skrive ut et linjeskift. I C++ kan også funksjonen `endl` brukes.

```
print(2+3)
print("Hello world!")
print("2+5=", 2+5)
```

Python

```
cout << 2+3 << endl;
cout << "Hello world!" << endl;
cout << "2+5=" << (2+5) << endl;
```

C++

Variabler

I Python kan man introdusere variabler bare ved å tilordne et variabelnavn en verdi. I C++ må vi første gang en variabel brukes deklarerer den ved å skrive datatypen foran variabelnavnet. Det er vanlig å tilordne verdier til variabler samtidig som vi deklarerer dem (initialisering), men det er ikke påbudt. En del moderne programmeringsomgivelser gir deg advarsel om variable ikke initialiseres.

```
a = 1
b = 2
c = a + b
d = c // b
print(d)
d = d * b
print(d)
```

```
e = c
f = e / 2;
print(f)
```

```
g = c / b
print(g)
```

```
h = a%b
print(h)
```

Python

```
int a = 1;
int b = 2;
int c = a + b;
int d = c / b;
cout << d << endl;
d = d * b;
cout << d << endl;
```

```
double e = c;
double f = e / 2.0;
cout << f << endl;
```

```
double g = c / static_cast<double>(b);
cout << g << endl;
```

```
int h = a%b;
cout << h << endl;
```

C++

Øverst i Python-eksempelet ser vi at tallene 1 og 2 henholdsvis tilordnes variablene `a` og `b`. Både 1 og 2 er skrevet som heltall (ingen komma), og `a` og `b` vil derfor automatisk bli tolket som heltall av Python. I C++ kreves det at vi eksplisitt spesifiserer typen til variabelen i koden.

Vi har i dette eksempelet sett på to forskjellige datatyper, `int` og `double`. `int` er en type som kun kan representere heltall. En variabel av typen `int` opptar ofte (men ikke alltid) 32 bits i minnet, og kan dermed representere heltall mellom -2^{31} og $2^{31} - 1$. `double` er en type som inneholder *flyttall*, som brukes til å representere reelle tall. Ulikt en variabel av typen `int` er en variabel av typen `double` vanligvis 64 bits stor, og har en presisjon på mellom 15 og 17 sifre avhengig av størrelsen på tallet. En ting som er verdt å merke seg er at flyttall ikke kan representere *alle* reelle tall (da det er et uendelig antall av dem i alle intervaller). (En må derfor være klar over at bruk av flyttall normalt vil føre til større eller mindre «avrundingsfeil». Hvis en har to ulike flyttallsberegninger som rent matematisk skal gi eksakt samme verdi, så kan de på en datamaskin ofte bli ulike. Når en tester på likhet av flyttalsverdier kan en derfor være nødt til å legge inn en feilmargin.)

På linje 4 ser vi forskjellen mellom divisjonsoperatorene i Python og C++. I C++ gir den vanlige divisjonsoperatoren `/` *heltallsdivisjon* når den blir brukt til å dele to heltall. Dette betyr at svaret av divisjonen blir rundet ned til nærmeste heltall (hvis svaret er positivt), eller opp til nærmeste heltall dersom svaret er negativt (altså alltid "mot null"). Dette tilsvarer operatoren `//` i Python. Dersom man ønsker å få et desimaltall (flyttall) som svar fra en divisjon mellom to heltall, er man nødt til å *caste* det ene tallet til en flyttallstype, f.eks. `double`, og så utføre divisjonen. Å *caste* en variabel betyr å gjøre den om til en annen type. Caster man for eksempel heltallet 4 til `double`, vil man få flyttallet 4.0. Caster man et flyttall, for eksempel 4.5 til `int`, vil man få heltallet 4. Et eksempel på casting er vist til slutt i figuren over. Det finnes ulike former for casting, men foreløpig trenger du bare å bry deg om `static_cast` som er en eksplisitt måte å be om en type-konvertering. Det er nevnt i læreboka §8.5.7 og også dekket av forelesning.

Input fra bruker

Python kan ta inn data fra brukeren ved hjelp av `input()`. I C++ bruker man i stedet objektet `cin` ("character input", også kalt "standard input stream"), som fungerer på en lignende måte som utskrift til skjerm med `cout`.

```
i = input("Skriv inn et tall: ")
j = input("Skriv inn et tall: ")
print("Summen av de to tallene: ", float(i)+float(j), sep='')
```

Python

```
double i = 0.0;
double j = 0.0;
cout << "Skriv inn et tall: ";
cin >> i;
cout << "Skriv inn et tall: ";
cin >> j;
cout << "Summen av de to tallene: " << (i+j) << endl;
```

C++

Python lar oss skrive ut en forespørsel til bruker om hva som skal skrives inn, mens vi i C++ vil måtte gjøre dette i to steg, først en utskrift, og så en forespørsel om input. Ved bruk av `>>`-operatoren tar C++ seg av å lese inn og tolke verdiens type. Dette fungerer slik at verdien som leses inn vil bli tolket som å ha samme type som variabelen den skal lagres i, altså `double` i eksempelet over.

If-setninger

En if-test ser slik ut i Python og C++:

```
b = 2
if b > 2:
    print("B is greater than 2")
else:
    print("B is less than or equal to 2")
```

Python

```
int b = 2;
if (b > 2) {
    cout << "B is greater than 2" << endl;
} else {
    cout << "B is less than or equal to 2" << endl;
}
```

C++

Store likhetstrekk i syntaksen og kontrollstrukturen fungerer som forventet.

For-løkker

En enkel **for**-løkke som kjører fra 1 til 10 ser slik ut i Python og C++:

<pre>for i in range(1, 10+1): print(i)</pre>	<pre>for (int i = 1; i < 10+1; ++i) { cout << i << endl; }</pre>
Python	C++

Som du legger merke til så ser **for** i C++ litt annerledes ut, men vi finner igjen 1 og 10 som grenser også her. At vi har en steglengde på 1 er litt mindre intuitivt. Dette kommer av den siste delen av argumentet til **for**-løkken, **++i**, som inkrementerer verdien av variabelen **i**. I eksempelet under brukes steglengde 2 i stedet, noe som fører til at kun oddetallene skrives ut:

<pre>for i in range(1,10+1,2): print(i)</pre>	<pre>for (int i = 1; i < 10+1; i = i + 2) { cout << i << endl; }</pre>
Python	C++

Uttrykk på formen **i = i + x** skrives ofte som **i += x** i C++. I vårt tilfelle kunne vi altså i stedet skrevet **i += 2**. Legg også merke til at vi uttrykker lengden på **for**-løkka vår som en sammenligning, **i < 10+1**. Dette fungerer slik at **for**-løkken fortsetter så lenge denne sammenligningen er sann. Her kunne vi skrevet **i <= 10** i stedet for **i < 10+1**. Det er i C++ normalt å starte løkkene på 0 og dermed bruke **<** i stedet for **<=**, blant annet fordi indeksen til vectorer (og tabeller) i C++ starter på 0, på samme måte som i Python.

While-løkker

<pre>i = 1 while i < 1000: i *= 2 print(i)</pre>	<pre>int i = 1; while (i < 1000) { i *= 2; cout << i << endl; }</pre>
Python	C++

Igjen er det kun mindre forskjeller i syntaks mellom en **while**-løkke i Python og en i C++.

Funksjoner

<pre>def get_four(seed): return 4</pre>	<pre>int getFour(int seed) { return 4; }</pre>
Python	C++

I Python defineres funksjoner ved å skrive **def funksjonsnavn(parametre)**. Dette er ulikt måten vi definerer funksjoner på i C++, der vi også må ha med *typen til funksjonens returverdi*. Hvis en funksjon ikke har noen retur-verdi deklarerer vi retur-"typen" som **void**. Som når variabler vanligvis defineres må funksjonens argumenter defineres med typer. I dette tilfellet ser vi at funksjonen tar inn et heltall, **int**.

Eksponensierings-operatoren

I Python bruker vi operatoren `**` for å eksponensiere ("opphøye") et tall. I C++ finnes ikke denne, men vi har to andre alternativer. For enkle uttrykk, der eksponenten er et lite heltall, kan vi eksplisitt multiplisere tallet vi vil eksponensiere med seg selv, f.eks. $x^3 = x * x * x$. Alternativt kan vi benytte funksjonen `pow`.

```
#include "std_lib_facilities.h"

int main() {
    int fourSquared = pow(4,2);
    int fourCubed = pow(4,3);
    cout << "4^2: " << fourSquared
         << " 4^3: " << fourCubed << endl;
    return 0;
}
```

Eksempel på eksponensiering ved hjelp av `pow` i C++.

I dette eksempelet inkluderer vi filen `std_lib_facilities.h`. **Du får tilgang til denne ved å følge fremgangsmåten beskrevet i øving 0 når du oppretter ett nytt prosjekt.** En slik fil kalles ofte en include-fil eller en header-fil. Denne filen benyttes i mange av øvingene for å redusere mengden av detaljer en må huske på å ha på plass i programmet for å få til en øving. I dette tilfellet gir headerfilen oss tilgang til funksjonen `pow`. `pow(x,n)` er en innebygget funksjon i biblioteket som beregner og returnerer verdien x^n (Se evt. læreboka §15.5 side 532).

Uttrykket med `cout` går over to linjer. Dersom en linje ikke ender med semikolon, vil kompilatoren forsøke å tolke neste linje som en fortsettelse av program-setningen (her et uttrykk). Ofte separerer man logisk separate deler av lange program-setninger på denne måten, slik at koden blir lettere å lese og linjene ikke blir for lange.

1 Kodeforståelse: oversett til Python(10%)

a) Oversett følgende kodesnutt til Python.

```
bool isFibonacciNumber(int n) {
    int a = 0;
    int b = 1;
    while (b < n) {
        int temp = b;
        b += a;
        a = temp;
    }
    return b == n;
}
```

Skriv løsningen din i **INGInious**. Når du skriver koden i Python, husk at innrykk er viktig for å angi hvilke kodelinjer som hører til hvilke kontrollstrukturer (if-setninger, løkker, funksjoner osv). Bruk enten tabs eller 4 spaces for innrykk, ikke en blanding.

Nyttig å vite: Organisering og kjøring av kode

I øvingene i dette faget er det ikke nødvendig med mer enn **ett prosjekt per øving** i utviklingsmiljøet ditt (VS Code). I denne øvingen trenger du bare én *kildefil* (.cpp-fil) i prosjektet. Filene dere trenger vil som regel bli utdelt som en del av skjelettkode til hver øving. Denne skal inneholde alle funksjonene i Oppgave 2. Har du fullført øving 0 skal du nå ha en ferdig prosjekt-mal for et slikt prosjekt, som gjør det raskt og enkelt å opprette et prosjekt av denne typen.

For at programmer ikke skal bli uoversiktlige deler man dem opp i funksjoner, som inneholder gjenbrukbar kode. Denne koden kan bli kjørt fra andre steder i programmet ved hjelp av et funksjonskall. (Alle programmer har minst en funksjon. Når programmet startes vil operativsystemet kjøre funksjonen som heter `main`.)

For å teste funksjonene du har laget kan du kalle disse fra `main`, som i eksempelet under.

```
#include "std_lib_facilities.h"

// funksjonen 'add' som har to parameter av
// typen 'int' og returverdi av typen 'int'.
int add(int a, int b) {
    // legger sammen verdiene av 'a' og 'b' og returner resultatet
    return a + b;
}

int main() {
    // Skriver 'Hello world!' i konsollen
    cout << "Hello world!" << endl;

    // Kaller på 'add' med heltallene 1 og 2 som argumenter.
    // Returverdien (3) skrives ut i konsollen
    cout << add(1, 2) << endl;

    return 0;
}
```

2 Oversett fra Python til C++ (90%)

Oversett følgende kodesnutter til C++, og sjekk at de både kompilerer og kjører.

Fra nå av skal skjelettkode benyttes som utgangspunkt for å løse oppgavene. Skjelettkoden hentes via VS Code-utvidelsen TDT4102 Tools. Åpne kommandopaletten med **Ctrl+Shift+P** (**Cmd+Shift+P** om du er på Mac) → **Create project from template** → **Exercises** → **001**. For hver deloppgave vil man finne to kommentarer som definerer henholdsvis begynnelsen og slutten av svaret ditt. Kommentarene er på formatet: `// BEGIN: <oppgavenummer><deloppgave>` og `// END: <oppgavenummer><deloppgave>`.

I oppgave 2a skriver man mellom `// BEGIN: 2a` og `// END: 2a`. I oppgave 2b skriver man mellom `// BEGIN: 2b` og `// END: 2b` osv. Kun kode som blir skrevet innenfor kommentarene rettes. Det er viktig at ikke `BEGIN`- eller `END`-kommentarene fjernes da dette fører til at oppgaven ikke blir automatisk rettet og 0 poeng blir gitt.

Det går fint å skrive kode utenfor disse blokkene for sin egen del, for eksempel for å teste funksjoner i `main`, men dette vil ikke tas med i rettingen.

a) Implementer funksjonen `maxOfTwo`

Denne funksjonen tar inn to heltall, A og B, skriver ut om A er større enn B eller om B er større eller lik A, og returnerer det største tallet.

```
def maxOfTwo(a, b):
    if a > b:
        print("A is greater than B")
        return a
    else:
        print("B is greater than or equal to A")
        return b
```

b) `main`-funksjonen

- I `main`-funksjonen legg til følgende kodesnutt:


```
cout << "Oppgave a)" << endl;
cout << maxOfTwo(5, 6) << endl;
```
- Sørg for at koden kompilerer, og gir forventet output. Som er følgende:


```
Oppgave a)
B is greater than or equal to A
6
```
- For alle de resterende deloppgavene, lag tilsvarende testkode i `main`-funksjonen etter at du har oversatt funksjonen(e) og sørg for at du får forventet output.

c) Implementer funksjonen `fibonacci`

Denne funksjonen tar inn et heltall n, og skriver ut de n første Fibonacci-tallene, og returnerer Fibonacci-tall nummer n+1.

```
def fibonacci(n):
    a = 0
    b = 1
    print("Fibonacci numbers:")
    for x in range(1, n + 1):
        print(x, b)
        temp = b
        b += a
        a = temp
    print("----")
    return b
```


d) Implementer funksjonen squareNumberSum

Denne funksjonen tar inn et heltall n , skriver ut de n første kvadrat-tallene, og både printer og returnerer summen av disse.

```
def squareNumberSum(n):
    totalSum = 0
    for i in range(1, n + 1):
        totalSum += i * i
        print(i * i)
    print(totalSum)
    return totalSum
```

e) Implementer funksjonen triangleNumbersBelow

Denne funksjonen tar inn et heltall n , og printer alle trekant-tallene¹ under n .

```
def triangleNumbersBelow(n):
    acc = 1
    num = 2
    print("Triangle numbers below ", n, ":", sep="")
    while acc < n:
        print(acc)
        acc += num
        num += 1
    print()
```

Er det mulig å skrive returverdien til en void-funksjon til skjerm?

f) Implementer funksjonen isPrime

Denne funksjonen tar inn et heltall n , og returnerer True dersom n er et primtall, og False dersom n ikke er det.

```
def isPrime(n):
    for j in range(2, n):
        if n % j == 0:
            return False
    return True
```

g) implementer funksjonen naivePrimeNumberSearch

Denne funksjonen tar inn et heltall n , og printer ut alle primtallene under n etterfulgt av stringen "is a prime".

```
def naivePrimeNumberSearch(n):
    for number in range(2, n):
        if isPrime(number):
            print(number, " is a prime")
```

h) Implementer funksjonen inputAndPrintNameAndAge

Denne funksjonen ber brukeren om navn og alder og skriver det ut.

```
def inputAndPrintNameAndAge():
    name = input("Skriv inn et navn: ")
    age = input("Skriv inn alderen til " + name + ": ")
    print(name + " er " + age + " år gammel.")
```

Har du husket å teste alle funksjonene? Vi vil også nevne at du generelt, og i enda større grad i de kommende øvinger **bør teste koden din etter hvert som du skriver den**. Det er mye lettere og mindre tidkrevende å finne feil i et lite program enn et stort, og stegvis utvidelse av et program er en anerkjent arbeidsmåte.

¹<https://no.wikipedia.org/wiki/Trekanttall>

Kjøreeksempel

Eksemplene i dette avsnittet er veiledende, og tar utgangspunkt i lignende kjøring som oppgave 2a).

2c)

Input:

```
cout << "Oppgave c)" << endl;
cout << fibonacci(5) << endl;
```

Terminal:

```
Oppgave c)
Fibonacci numbers:
1 1
2 1
3 2
4 3
5 5
----
8
```

2d)

Input:

```
cout << "Oppgave d)" << endl;
cout << squareNumberSum(5) << endl;
```

Terminal:

```
Oppgave d)
1
4
9
16
25
55
55
```

2e)

Input:

```
cout << "Oppgave e)" << endl;
triangleNumbersBelow(10);
```

Terminal:

```
Oppgave e)
Triangle numbers below 10:
1
3
6
```

2f) og 2g)

Input:

```
cout << "Oppgave f) og g)" << endl;
naivePrimeNumberSearch(14);
```

Terminal:

```
Oppgave f) og g)
2 is a prime
3 is a prime
5 is a prime
7 is a prime
11 is a prime
13 is a prime
```

2h)

Input:

```
inputAndPrintNameAndAge();
```

Terminal:

```
Skriv inn et navn: Per
Skriv inn alderen til Per: 20
Per er 20 år gammel.
```